

# 컴퓨터 그래픽스 실습 개념 및 시험 완벽 대비

## 1. 그리기의 기초 (Primitive & Attributes)

### 1.1 선형 보간 (Linear Interpolation)과 직선

두 점 사이를 연결하는 직선은 수학적으로 비율( $t$ )에 따른 점들의 집합입니다.

[코드 예시: code/line\_points.cpp]

```
// 0.0에서 1.0까지 t를 증가시키며 중간 점들을 계산
for (int i = 0; i <= 20; ++i) {
    float t = i / 20.0f; // t: 0.0 ~ 1.0

    // 선형 보간 공식: P = Start + t * (End - Start)
    // 원점(0,0)에서 (50,50,50)으로 가는 경우:
    float x = 50.0f * t;
    float y = 50.0f * t;
    float z = 50.0f * t;

    glVertex3f(x, y, z);
}
```

- 해설:  $t$ 가 0이면 시작점, 1이면 끝점이 됩니다. 이  $t$ 를 이용한 보간 개념은 나중에 애니메이션 경로 이동이나 그라데이션 색상 계산에도 똑같이 쓰이는 핵심 원리입니다.

### 1.2 삼각함수와 나선(Spiral)

원은  $\sin$ ,  $\cos$ 으로 그리며, 여기에  $z$ 축 변화를 주면 나선이 됩니다.

```
z = -50.0f; // 시작 깊이

// 각도(angle)를 계속 증가시키며 회전
for(angle = 0.0f; angle <= (2.0f * 3.14f) * 3.0f; angle += 0.1f) {
    // 1. 원 그리기 공식
    x = 50.0f * cos(angle);
    y = 50.0f * sin(angle);

    // 2. 깊이(z)를 지속적으로 변경 -> 나선형 궤적 형성
    glVertex3f(x, y, z);
    z += 0.5f;
}
```

- 시험 포인트: 만약  $z += 0.5f$  대신 반지름(50.0f) 변수를  $r += 0.5f$ 처럼 증가시켰다면? → 평면상의 골뱅이 모양이 됩니다. `glPointSize`를 증가시켰다면? → 점이 점점 커지는 나선이 됩니다( code/spiral\_grow.cpp ).

## 2. 스타일링과 비트 패턴

### 2.1 점선 (Stipple) 패턴

점선은 그림이 아니라 비트 마스크입니다.

```

glEnable(GL_LINE_STIPPLE); // ★ 필수: 기능 활성화

// factor: 확대 배율 (3배 늘림)
// pattern: 0x00FF (0000 0000 1111 1111) -> 8칸 공백, 8칸 선
glLineStipple(3, 0x00FF);

glBegin(GL_LINES);
    glVertex3f(-80.0f, 0.0f, 0.0f);
    glVertex3f(80.0f, 0.0f, 0.0f);
glEnd();

```

- 해설: 패턴 0x00FF의 비트가 1인 곳은 그려지고 0인 곳은 비워집니다. factor가 3이면 비트 하나가 3픽셀만큼의 길이를 차지합니다.
- 애니메이션: 루프마다 `pattern = pattern * 16` 처럼 비트 연산을 하면 점선이 움직이는 효과를 낼 수 있습니다.

### 3. 3차원 변환과 상호작용 (★ 핵심)

가장 혼동하기 쉬운 변환 순서(Transformation Order)입니다.

```

glPushMatrix();

// [코드 작성 순서] 1. 회전 -> 2. 이동
glRotatef(xRot, 1.0f, 0.0f, 0.0f); // X축 회전
glRotatef(yRot, 0.0f, 1.0f, 0.0f); // Y축 회전
glTranslatef(xTran, yTran, 0.0f); // 이동

// [실제 동작 (Local 좌표계 기준)]
// 물체는 자신의 축을 기준으로 '회전'을 먼저 한 상태에서,
// 그 회전된 방향이 아니라 '원래 축 방향'으로 이동합니다.
// (직관적으로는: 제자리 회전 후 위치 이동)

DrawPizza(); // 물체 그리기

glPopMatrix();

```

- 주의: 만약 `glTranslatef`를 먼저 쓰고 `glRotatef`를 썼다면? → 물체가 원점 주위를 공전(Orbit)하게 됩니다.
- 상호작용: `xRot`, `yRot` 변수는 `glutSpecialFunc` (방향키)에서 변경되며, 반드시 `glutPostRedisplay()`를 호출해야 `RenderScene`이 다시 실행되어 화면이 바뀝니다.

### 4. 가시성 판단과 깊이 처리 (★ 시험 최빈출)

"어떤 면을 그리고, 어떤 면을 버릴 것인가?"는 성능과 올바른 렌더링을 위해 필수적입니다.

#### 4.1 후면 제거 (Back-face Culling)

앞면과 뒷면을 구분하여, 보이지 않는 뒷면은 그리지 않습니다.

```

glEnable(GL_CULL_FACE); // 후면 제거 활성화 (기본: CCW가 앞면)

// 1. 앞면 그리기 (반시계 방향 - CCW)
// 카메라가 볼 때 정상적으로 보임
glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f); // 중심
    for (...) {
        // 반시계 방향으로 정점 생성
    }

```

```

        glVertex2f(x, y);
    }
    glEnd();

// 2. 뒷면 그리기 (시계 방향 - CW)
// ★ 중요: 일부러 시계 방향(CW)으로 그려서 '뒷면'임을 명시함.
// 이렇게 하면 정면에서는 안 보이다가(Culling 됨),
// 피자가 회전하여 뒷면이 드러날 때(카메라 입장에서 CCW가 될 때) 보임.
glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f);
    for (angle = 2.0f * PI; ... ) { // 각도를 거꾸로 돌림
        // 시계 방향 생성
    }
    glEnd();

```

- 해설: 만약 뒷면도 반시계 방향(CCW)으로 그렸다면? → 앞면과 똑같이 취급되어, 피자 뒷면이 투시되거나 겹쳐서 이상하게 보일 것입니다. 뒷면은 반드시 정점 순서를 반대로 하거나 논리적으로 뒤집어야 합니다.

## 5. 카메라와 투영 (★ 함정 문제 주의)

`gluLookAt` 과 물체 위치의 상관관계를 묻는 문제는 반드시 나옵니다.

```

// 1. 투영 설정 (함정 주의)
// near(-100) < far(100) 이므로 정상적인 Z축 (앞이 -Z, 뒤가 +Z 아님에 주의... OpenGL은 -Z가 앞)
// 정확히는: 카메라는 -Z를 바라보고, 물체는 그 앞에 배치됨.
glOrtho(-100, 100, -100, 100, -100, 100);

// 2. 카메라 설정 (gluLookAt)
gluLookAt(0.0f, 0.0f, 0.0f, // Eye: 원점
          0.0f, 0.0f, 1.0f, // Center: +Z 방향을 바라봄 (특이 케이스)
          0.0f, 1.0f, 0.0f); // Up

// 3. 물체 배치
// Blue 원: z = 10 (카메라에서 거리 10)
// Red 원: z = 20 (카메라에서 거리 20)

```

- 분석:
  - 카메라는 (0,0,0)에서 +Z 쪽을 보고 있습니다.
  - 물체들은  $Z = 10, 20$ 인 양수 좌표에 있습니다. 즉, 카메라가 보는 방향에 물체가 있습니다.
  - 결과: 거리 10인 Blue 원이 거리 20인 Red 원보다 앞에 있습니다. 따라서 **Blue가 Red를 가립니다**.
  - 만약: `gluLookAt(..., 0,0,-1, ...)` 로 기본값(-Z 보기)이었다면? → 물체들은 카메라 뒤통수에 있으므로, `glOrtho` 범위 안이라도 화면에 안 나올 수 있거나(Culling 등), 논리적으로 뒤에 있는 것으로 처리됩니다.

## 6. 셰이딩 (Shading Model)

면의 색상을 채우는 방식입니다.

```

// GL_FLAT: 마지막 정점(Blue) 색 하나로 삼각형 전체를 칠함 (단색)
// GL_SMOOTH: Red -> Green -> Blue로 부드럽게 섞임 (그래데이션)
glShadeModel(GL_SMOOTH);

glBegin(GL_TRIANGLES);
    glColor3f(1.0f, 0.0f, 0.0f); glVertex2f(0.0f, 0.0f); // Red
    glColor3f(0.0f, 1.0f, 0.0f); glVertex2f(50.0f, 0.0f); // Green

```

```
glColor3f(0.0f, 0.0f, 1.0f); glVertex2f(50.0f, 50.0f); // Blue  
glEnd();
```

- 해설: GL\_SMOOTH 는 정점 사이의 색상을 선형 보간(1.1절 참고) 하여 채웁니다. 입체감을 줄 때 필수적입니다.